

0 Instructions

Homework is not meant to be turned in. Please refer to <https://courses.grainger.illinois.edu/cs446/sp2026> for course policy on homeworks. Use this homework to help you study for quizzes and exams, but it is not a comprehensive or sufficient study tool.

1 Contrastive and Self-Supervised Learning

1.1 Computing InfoNCE with Normalized Features

In contrastive learning, the InfoNCE loss encourages representations of positive pairs to be similar while pushing apart negative pairs. Let an encoder $f(\cdot)$ map inputs into a representation space, and define the similarity score using cosine similarity:

$$s(f(\mathbf{x}), f(\mathbf{x}')) = \frac{f(\mathbf{x})^T f(\mathbf{x}')}{\|f(\mathbf{x})\|_2 \|f(\mathbf{x}')\|_2}.$$

Suppose we have two images $\mathbf{x}^{(1)}$ and $\mathbf{x}^{(2)}$, and we generate augmented views $\tilde{\mathbf{x}}^{(1)}$ and $\tilde{\mathbf{x}}^{(2)}$. Assume each encoded feature vector below has norm 1, so cosine similarity reduces to the dot product. The feature vectors are:

$$f(\mathbf{x}^{(1)}) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad f(\tilde{\mathbf{x}}^{(1)}) = \begin{bmatrix} \frac{\sqrt{3}}{2} \\ \frac{1}{2} \end{bmatrix}, \quad f(\mathbf{x}^{(2)}) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad f(\tilde{\mathbf{x}}^{(2)}) = \begin{bmatrix} -\frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{bmatrix}.$$

Using $\mathbf{x}^{(1)}$ as the anchor, $\tilde{\mathbf{x}}^{(1)}$ as the positive sample, and $\mathbf{x}^{(2)}, \tilde{\mathbf{x}}^{(2)}$ as the negative samples, answer the following:

1. Compute the following similarity scores:

$$s(f(\mathbf{x}^{(1)}), f(\tilde{\mathbf{x}}^{(1)})), \quad s(f(\mathbf{x}^{(1)}), f(\mathbf{x}^{(2)})), \quad s(f(\mathbf{x}^{(1)}), f(\tilde{\mathbf{x}}^{(2)})).$$

2. Now compare the InfoNCE loss at temperature $\tau = 1$ versus a smaller temperature such as $\tau = \frac{1}{2}$. Using your answers from part (1), determine whether the loss increases or decreases when τ is reduced, and briefly explain why. You do not need to compute the exact numerical values, and you may leave your answer in terms of $\exp(\cdot)$.

The single-example InfoNCE loss is

$$\mathcal{L}_{\text{NCE}} = -\log \frac{\exp(s(f(\mathbf{x}^{(1)}), f(\tilde{\mathbf{x}}^{(1)}))/\tau)}{\exp(s(f(\mathbf{x}^{(1)}), f(\tilde{\mathbf{x}}^{(1)}))/\tau) + \exp(s(f(\mathbf{x}^{(1)}), f(\mathbf{x}^{(2)}))/\tau) + \exp(s(f(\mathbf{x}^{(1)}), f(\tilde{\mathbf{x}}^{(2)}))/\tau)}.$$

1.2 Dual-Encoder Contrastive Learning

In lecture, we defined the InfoNCE loss for self-supervised contrastive learning as

$$\mathcal{L}_{\text{NCE}} := -\mathbb{E} \left[\log \frac{\exp(s(f(\mathbf{x}), f(\mathbf{x}^+)))}{\exp(s(f(\mathbf{x}), f(\mathbf{x}^+))) + \sum_{i=1}^N \exp(s(f(\mathbf{x}), f(\mathbf{x}^{-(i)})))} \right],$$

where the score function $s(\mathbf{u}, \mathbf{v})$ is typically the cosine similarity or dot product.

We also discussed Contrastive Language-Image Pre-training (CLIP). As shown in Figure 1, CLIP uses a dual-encoder architecture to match images with text. Formally, for two domains $\mathcal{X}_1$ and $\mathcal{X}_2$ with encoders

$$f_1 : \mathcal{X}_1 \rightarrow \mathbb{R}^d, \quad f_2 : \mathcal{X}_2 \rightarrow \mathbb{R}^d,$$

and a paired sample $(\mathbf{x}, z) \in \mathcal{X}_1 \times \mathcal{X}_2$, assume the temperature is fixed to $\tau = 1$. Then a dual-encoder contrastive loss can be written as

$$\mathcal{L}_{\text{dual}}(\mathbf{x}, z) = -\log \frac{\exp(f_1(\mathbf{x})^T f_2(z))}{\sum_{j=1}^C \exp(f_1(\mathbf{x})^T f_2(z_j))}.$$

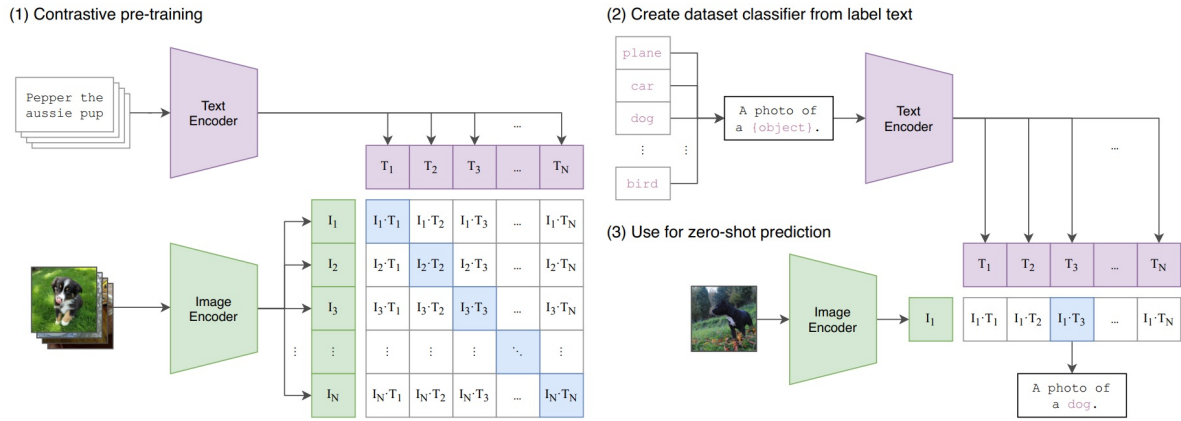


Figure 1: Summary of CLIP model's approach. From Learning Transferable Visual Models From Natural Language Supervision paper.

Now consider a standard labeled dataset

$$\mathcal{D} = \{(\mathbf{x}^{(k)}, y^{(k)})\}_{k=1}^K,$$

where $\mathbf{x}^{(k)} \in \mathcal{X}$ is an image and $y^{(k)} \in \{1, 2, \dots, C\}$ is its class label. A standard neural network classifier $g(\mathbf{x})$ consists of a feature extractor $h(\mathbf{x}) \in \mathbb{R}^d$ and a final linear layer with

Homework 4

weight matrix $W \in \mathbb{R}^{d \times C}$ (ignoring bias). The cross-entropy loss for a single sample is

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(\mathbf{w}_{y^{(k)}}^T h(\mathbf{x}^{(k)}))}{\sum_{c=1}^C \exp(\mathbf{w}_c^T h(\mathbf{x}^{(k)}))},$$

where $\mathbf{w}_c$ denotes the c -th column of W .

1. Show that supervised cross-entropy classification is a special case of dual-encoder contrastive learning. Specifically, define $\mathcal{X}_1$, $\mathcal{X}_2$, f_1 , f_2 , the anchor $\mathbf{x}$, the positive target z , and the candidate set $\{z_j\}_{j=1}^C$ so that $\mathcal{L}_{\text{dual}}$ becomes mathematically identical to $\mathcal{L}_{\text{CE}}$.
2. CLIP can perform zero-shot prediction, while a standard classifier with weight matrix W cannot classify a completely new class without retraining. Using your formulation from Question 1, explain why. Focus on the structural difference between f_2 in standard classification and f_2 in CLIP.

2 Large Language Models

2.1 Position Encoding

Positional encoding in Transformers enables the model to understand sequence order by adding unique position-dependent patterns to token embeddings. One positional encoding method uses sinusoidal functions of varying frequencies that create distinctive “fingerprints” for each position, allowing the model to identify both absolute and relative positions. The mathematical properties of these sinusoidal functions also permit the model to generalize to sequence lengths beyond training.

1. Write the equations for the sinusoidal positional encoding used in the Transformer paper *Attention Is All You Need*, given the token position pos , frequency index i , and token representation dimension d_{model} , where each token embedding and each positional encoding vector lies in $\mathbb{R}^{d_{\text{model}}}$.
2. Let $\text{PE}(\text{pos})$ denote the positional encoding and define

$$w_i = \frac{1}{10000^{2i/d_{\text{model}}}}.$$

Prove that $\text{PE}(\text{pos} + k)$ can be written as a linear function of $\text{PE}(\text{pos})$.

Hint:

$$\begin{aligned}\text{PE}_{(\text{pos}+k, 2i)} &= a \cdot \text{PE}_{(\text{pos}, 2i)} + b \cdot \text{PE}_{(\text{pos}, 2i+1)} \\ \text{PE}_{(\text{pos}+k, 2i+1)} &= c \cdot \text{PE}_{(\text{pos}, 2i)} + d \cdot \text{PE}_{(\text{pos}, 2i+1)}\end{aligned}$$

3. Let $\text{pos}_2 = \text{pos}_1 + k$. Show that the dot product

$$\text{PE}(\text{pos}_1)^\top \text{PE}(\text{pos}_2)$$

depends only on k (relative position), not on absolute positions.

2.2 Attention Basics

Consider a single attention head with query $\mathbf{q} \in \mathbb{R}^3$, keys

$$\mathbf{k}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{k}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{k}_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix},$$

and values

$$\mathbf{v}_1 = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 3 \\ 1 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix}.$$

The attention output is defined as

$$\mathbf{y} = \sum_{i=1}^3 \alpha_i \mathbf{v}_i, \quad \alpha_i = \frac{\exp(\mathbf{k}_i^\top \mathbf{q})}{\sum_{j=1}^3 \exp(\mathbf{k}_j^\top \mathbf{q})}.$$

We will evaluate attention for two different queries,

$$\mathbf{q}^{(1)} = \begin{bmatrix} 6 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{q}^{(2)} = \begin{bmatrix} 1 \\ 1 \\ 6 \end{bmatrix}.$$

For each query, compute the attention scores, attention weights, and the final attention output $\mathbf{y}$. Then compare the two results and explain how the query affects the attention weights and the final output. Based on this example, describe in one sentence what properties of the inputs to the attention operation would result in the output being approximately equal to one particular value vector $\mathbf{v}_j$ for some $j \in \{1, 2, 3\}$. Specifically, what must be true about the query $\mathbf{q}$, the values $\{\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3\}$, and/or the keys $\{\mathbf{k}_1, \mathbf{k}_2, \mathbf{k}_3\}$?

2.3 Multi-Head Attention

Recall that attention on query $\mathbf{q} \in \mathbb{R}^d$, keys $\{\mathbf{k}_1, \dots, \mathbf{k}_n\}$, and values $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ is defined by

$$\mathbf{y} = \sum_{i=1}^n \alpha_i \mathbf{v}_i, \quad \alpha_i = \frac{\exp(\mathbf{k}_i^\top \mathbf{q})}{\sum_{j=1}^n \exp(\mathbf{k}_j^\top \mathbf{q})}.$$

Assume the key vectors are mutually orthogonal and unit norm, i.e.

$$\mathbf{k}_i^\top \mathbf{k}_j = 0 \quad (i \neq j), \quad \|\mathbf{k}_i\| = 1.$$

Let $\mathbf{v}_a$ and $\mathbf{v}_b$ be two value vectors with corresponding keys $\mathbf{k}_a$ and $\mathbf{k}_b$.

1. Show that a single attention head can produce an output approximately equal to the average

$$\frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$$

by choosing an appropriate query vector $\mathbf{q}$. Give an expression for $\mathbf{q}$ in terms of $\mathbf{k}_a$ and $\mathbf{k}_b$, and briefly explain why it works.

2. Now suppose the keys are noisy. Specifically, each key is sampled from a normal distribution $\mathbf{k}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \Sigma_i)$, where the means $\boldsymbol{\mu}_i$ are mutually orthogonal and unit norm. Assume that for all $i \neq a$, the covariance is $\Sigma_i = \alpha \mathbf{I}$ for some small constant $\alpha > 0$. However, for key $\mathbf{k}_a$, the covariance is defined as

$$\Sigma_a = \alpha \mathbf{I} + \frac{1}{2} \boldsymbol{\mu}_a \boldsymbol{\mu}_a^\top.$$

As illustrated in Figure 2, this means that while the direction of $\mathbf{k}_a$ remains relatively stable, its *magnitude* fluctuates significantly along the $\boldsymbol{\mu}_a$ axis across different samples.

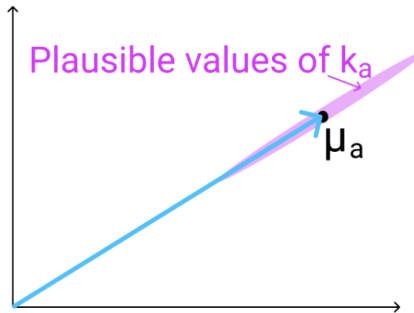


Figure 2: The vector $\boldsymbol{\mu}_a$ (shown here in 2D as an example), with the range of possible values of $\mathbf{k}_a$ shown in magenta. As mentioned previously, $\mathbf{k}_a$ points in roughly the same direction as $\boldsymbol{\mu}_a$, but may have larger or smaller magnitude.

Homework 4

If you use the same single-head query from Question 1, now defined as $\mathbf{q} = \lambda(\boldsymbol{\mu}_a + \boldsymbol{\mu}_b)$, describe qualitatively what happens to the attention output across different samples of the key vectors, especially as the sampled key $\mathbf{k}_a$ fluctuates while the other keys stay close to their means. Why does this geometric fluctuation make single-headed averaging brittle?

3. Consider a simple two-headed attention mechanism. Two queries $\mathbf{q}_1$ and $\mathbf{q}_2$ from the two heads produce outputs $\mathbf{y}_1$ and $\mathbf{y}_2$, and the final output is defined as the average of the outputs of the two heads:

$$\mathbf{y} = \frac{1}{2}(\mathbf{y}_1 + \mathbf{y}_2).$$

Choose $\mathbf{q}_1$ and $\mathbf{q}_2$ so that the final output is approximately

$$\frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$$

Then explain why this two-headed construction is more robust than the single-head construction in part (2).

2.4 Parameter Estimation in a Decoder-Only LLM

In this problem, we will estimate the number of trainable parameters in an **OLMo-style** decoder-only transformer. OLMo is a family of fully open language models released together with training data, code, and intermediate checkpoints, making it a useful concrete example for understanding modern LLM architectures. If you are curious about the broader project, see the OLMo paper. We will use a standard 2-layer MLP when counting parameters in each transformer block.¹

We will count parameters for a decoder-only transformer with the following notation:

- model embedding dimension D (each token is represented at every layer by a vector in $\mathbb{R}^D$; this is the width of the residual stream shown in Figure 3)
- number of transformer layers L
- number of attention heads N
- MLP hidden dimension F (inside feed-forward block, each token representation is first projected from $\mathbb{R}^D$ to an intermediate vector in $\mathbb{R}^F$ and then projected back to $\mathbb{R}^D$)
- vocabulary size V

Assume:

- standard multi-head self-attention,
- no bias terms in any linear layer,
- Inside each transformer block, a standard 2-layer MLP is used with one linear map from $\mathbb{R}^D$ to $\mathbb{R}^F$, followed by an element-wise nonlinearity, followed by one linear map from $\mathbb{R}^F$ back to $\mathbb{R}^D$,
- no learned positional embeddings, so positional encoding contributes no trainable parameters,
- non-parametric layer normalization, so layer norm has no trainable parameters,
- The weights of the input embedding layer and the output token prediction layer (“LM head”) are untied (independent).

¹The original OLMo architecture uses a SwiGLU MLP block rather than the simplified standard 2-layer MLP used in this problem. Very roughly, SwiGLU uses one projection to produce content features, another projection to produce a gate, applies the SiLU/Swish nonlinearity to the gate, and then multiplies them element-wise before projecting back down. If you are interested, you can see [here](#) for more implementation details, but this will not be required material for the quiz or exam.

Homework 4

We will instantiate the formulas using the following configuration:

$$D = 4096, \quad L = 32, \quad N = 32, \quad F = 11008, \quad V = 50304.$$

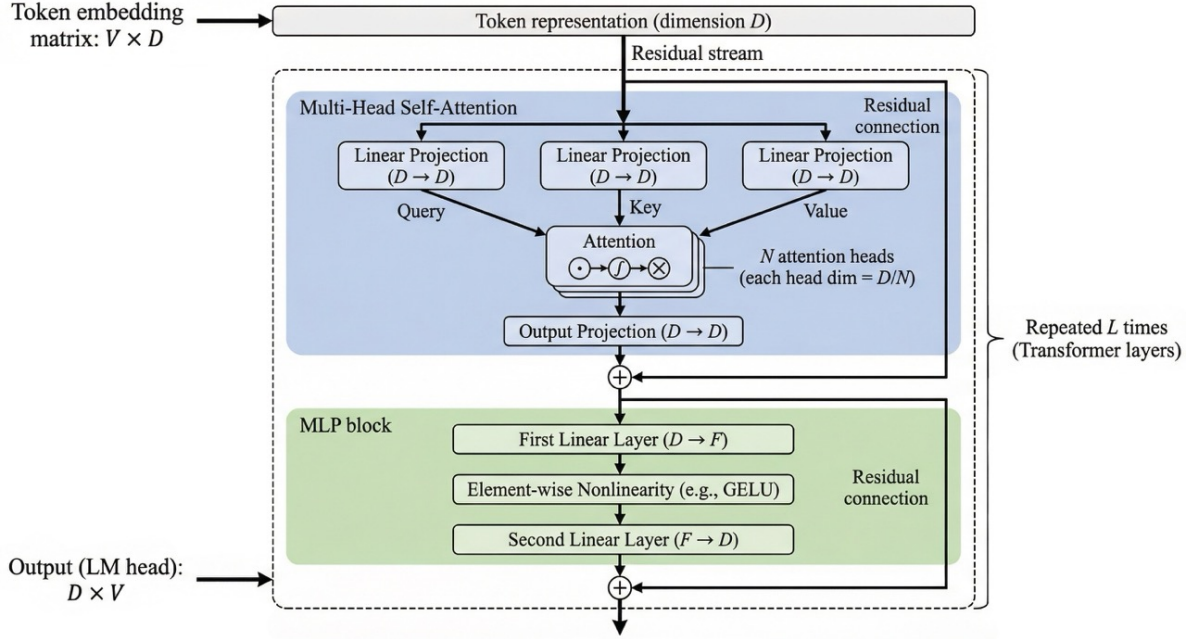


Figure 3: Detailed schematic of a decoder-only Transformer block. The diagram illustrates the data flow and tensor dimensional transformations from the input token representations through the Multi-Head Self-Attention and MLP blocks.

1. In standard multi-head self-attention, the trainable projection matrices are

$$W_Q, W_K, W_V, W_O.$$

Each attention head has dimension D/N . Derive the total number of trainable parameters in the attention block of one layer.

2. Consider the standard 2-layer MLP used in this problem. It first projects the token representation from dimension D up to dimension F , applies an element-wise nonlinearity, and then projects back down to dimension D . This block uses two learned matrices:

$$W_{\text{up}} \in \mathbb{R}^{D \times F}, \quad W_{\text{down}} \in \mathbb{R}^{F \times D}.$$

How many trainable parameters does each of W_{up} and W_{down} contain? Derive the total number of trainable parameters in the MLP block of one layer.

3. A full decoder-only transformer has:

- L identical transformer layers, each containing one attention block and one standard 2-layer MLP block,
- one input embedding matrix of shape $V \times D$,
- one output LM head of shape $D \times V$.

- (a) Derive a symbolic formula for the trainable parameters in one transformer layer.
- (b) Derive a symbolic formula for the total number of trainable parameters in the full decoder-only LLM.
- (c) Compute the total number of trainable parameters.

3 Diffusion Model

In the forward diffusion process, we gradually convert a data sample $\mathbf{x}_0 \sim q_0(\mathbf{x})$ by adding a small Gaussian noise in each step. We define

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}).$$

In the reverse diffusion process, we generate $\mathbf{x}_0$ by passing noise through $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$.

1. A diffusion model shares similarities with a VAE in that it also has an encoder stage (the forward diffusion process), which encodes a data sample $\mathbf{x}_0$ to noise $\mathbf{x}_T$, and a decoder stage (the backward diffusion process), which decodes the noise to a data sample. When training a diffusion model, we can also maximize the ELBO to optimize the log-likelihood $\mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)}[\log p_\theta(\mathbf{x}_0)]$. Write down the formula for the ELBO for $\log p_\theta(\mathbf{x}_0)$ in a diffusion model, and indicate which distribution the expectation is taken with respect to using the subscript of $\mathbb{E}[\cdot]$.
2. One way to evaluate generative models is to report the likelihood (or density) $p_\theta(\mathbf{x}_0)$ where $\mathbf{x}_0$ is sampled from the test set. Can we directly estimate the density in diffusion models?
3. Derive $q(\mathbf{x}_t | \mathbf{x}_0)$ as a function of β_i , $i = 1, 2, \dots, t$, and $\mathbf{x}_0$.
Hint: Use the reparameterization trick.
4. The training objective contains the KL-divergence term

$$\text{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)).$$

Derive the posterior mean $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0)$ of $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$.

Hint: Using Bayes' rule, $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{x}_0) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)}$.

Hint: $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ is also a Gaussian distribution.